

Student Academic Tracker & Grade System

A Lightweight, High-Precision Command-Line Application for Academic Portfolio Management, Credit-Weighted CGPA Computation, and Batch Performance Analytics.

Executive Abstract

This document serves as the comprehensive Engineering Report for the Student Academic Tracker & Grade System. The system addresses common institutional challenges in student transcript management, credit-weighted grade evaluation, and local record persistence. Built entirely in Python using core standard libraries, the software guarantees zero-dependency execution across modern operating systems, providing high computational reliability, fault-tolerant input validation, ASCII transcript rendering, and CSV report exports.

PROJECT TITLE

Student Academic Tracker

DOCUMENT TYPE

Detailed Project Report

CORE RUNTIME

Python 3.7+ (Zero Dependencies)

STORAGE ARCHITECTURE

JSON Database & CSV Reporting

SYSTEM INTERFACE

Interactive Command Line (CLI)

STATUS & VERSION

Production Release v1.0.0

1. INTRODUCTION

In modern educational administration, tracking student academic progress across multiple semesters requires accurate computation of cumulative performance metrics. The **Student Academic Tracker & Grade System** is a localized, command-line software solution designed to streamline student registration, course enrollment, credit-weighted grade calculations, and transcript generation.

Unlike complex cloud-based Enterprise Resource Planning (ERP) systems that demand constant network availability and heavy installation footprints, this application provides an instant, offline, zero-dependency environment. It equips academic advisors and students with an accessible tool to evaluate academic performance, analyze cohort metrics, and generate audit-ready reports.

2. PROBLEM STATEMENT

Educational departments and individual students frequently encounter specific operational bottlenecks when managing course records and computing academic standing:

- **Manual Calculation Errors:** Calculating Cumulative Grade Point Averages (CGPA) manually across varying course credit weights (\$1\$ to \$6\$ credits) is prone to human error, particularly during grade point rounding.
- **Over-Complex ERP Infrastructure:** Full-scale institutional software platforms often present steep learning curves, slow load times, and unnecessary complexity for basic departmental record tracking.
- **Fragile Spreadsheet Management:** Relying on basic spreadsheet templates often leads to accidental formula overrides, lack of strict input validation, and data corruption.
- **Lack of Local Data Persistence:** Basic terminal utilities often fail to save state between sessions, causing data loss upon exiting the application.

3. PROJECT SCOPE & TARGET USERS

In-Scope Functionality • Complete Student Profile CRUD lifecycle. • Credit-weighted course grade entry (0–100 marks). • Automated Letter Grade & Point conversion. • ASCII Transcript generation with CGPA. • JSON persistence & CSV export. • Cohort performance analytics (Mean, Max, Min).	Out-of-Scope Elements • Multi-tenant cloud server synchronization. • Graphical User Interface (GUI) desktop windowing. • Direct SQL server network connections. • Role-based active directory authentication.
--	--

Target User Personas

USER GROUP	PRIMARY OBJECTIVE	KEY SYSTEM BENEFIT USED
Academic Advisors	Maintain student course records & verify standing.	Formatted ASCII Transcripts & CSV Exports.
Department Heads	Analyze class cohort performance trends.	Batch Analytics Report (Mean, Max, Min GPA).
University Students	Track ongoing GPA and plan course credits.	Instant CGPA computation & course tracking.

4. FUNCTIONAL REQUIREMENTS

The functional requirements define the core operational capabilities that the system must execute reliably.

REQ ID	MODULE	REQUIREMENT DESCRIPTION	PRIORITY
FR-01	Student Registration	System shall allow registering new students with a unique ID, full name, and department.	High
FR-02	Directory Display	System shall display all registered students in a formatted tabular layout.	High
FR-03	Search Engine	System shall support case-insensitive searching by exact ID or partial student name.	Medium
FR-04	Record Deletion	System shall allow deleting student records after explicit user confirmation.	Medium
FR-05	Course Entry	System shall add/update course entries with course code, title, credits (1-6), and marks (0-100).	High
FR-06	Grade Conversion	System shall automatically map numerical marks to letter grades (S, A, B, C, D, E, F) and points.	High
FR-07	Transcript Generator	System shall render an ASCII academic transcript detailing course credits, points, and final CGPA.	High
FR-08	Batch Analytics	System shall compute cohort metrics: total enrolled, active count, average GPA, top/bottom performers.	Medium
FR-09	CSV Export	System shall export overall student metrics and CGPA summaries into <code>`academic_report.csv`</code> .	High
FR-10	JSON Persistence	System shall automatically load and save database state from/to <code>`students_data.json`</code> .	High

5. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements dictate the operational qualities, performance bounds, and usability constraints of the software environment.

Reliability & Fault Tolerance

The system must handle invalid inputs (letters in numeric prompts, negative credits, marks > 100) gracefully without throwing uncaught exceptions or terminating unexpectedly.

Portability & Zero Dependencies

The software must execute on standard Python 3.7+ runtimes on Windows, macOS, and Linux without requiring external ``pip`` package installations.

Execution Performance

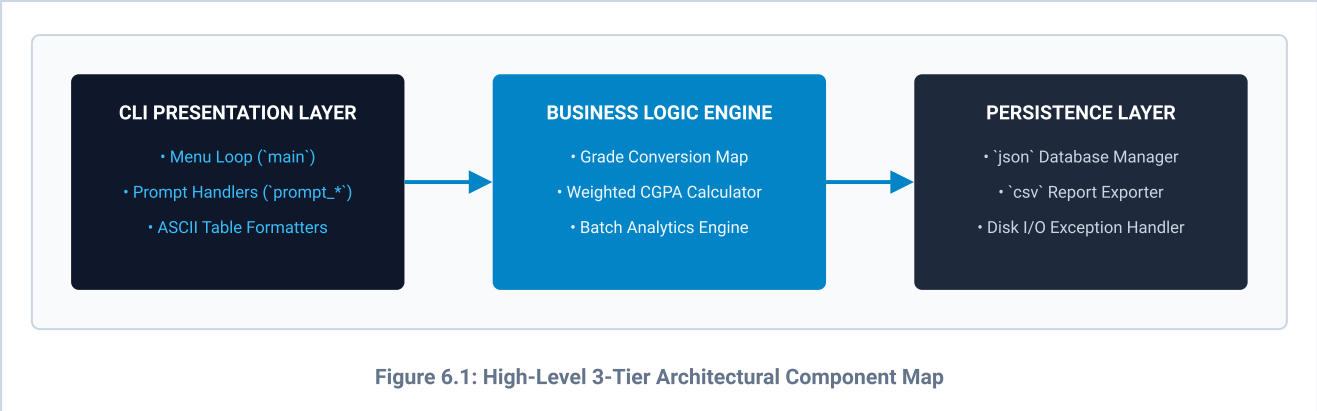
All menu navigation, grade computations, and local JSON operations must execute within < 100 milliseconds for datasets of up to 1,000 student records.

Data Integrity & Safety

Modifications to database records must automatically synchronize with local disk storage, and signal interrupts (``Ctrl+C``) must terminate safely without corrupting JSON files.

6. SYSTEM ARCHITECTURE

The software follows a modular, 3-tier architectural pattern adapted for command-line execution: the **Presentation (CLI View) Layer**, the **Controller / Business Logic Engine**, and the **Data Access & Persistence Layer**.



7. DESIGN DECISIONS & RATIONALE

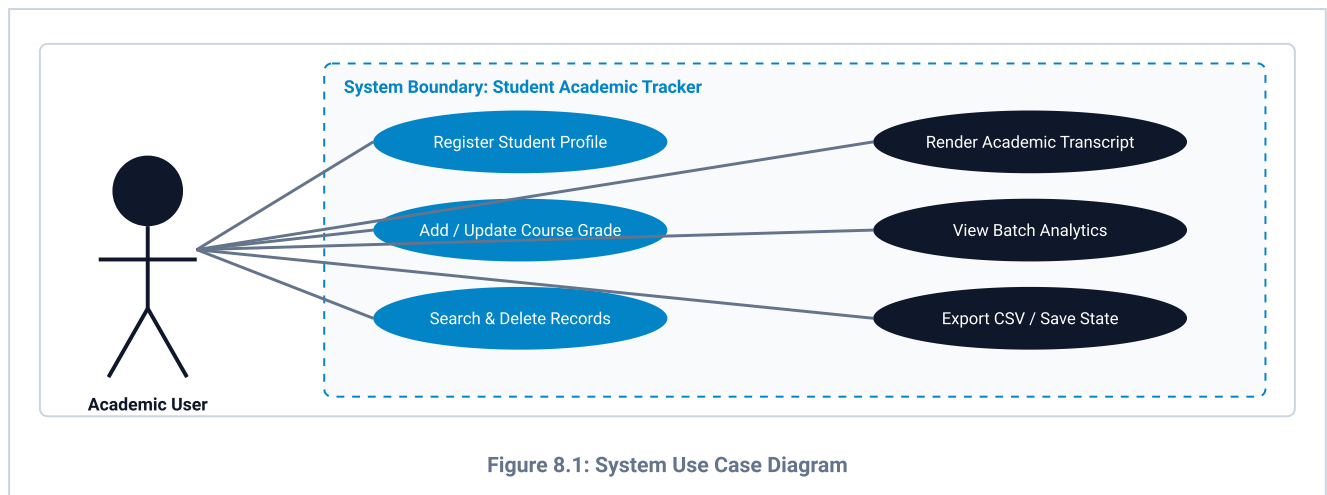
Every structural design choice was selected to balance simplicity, maintenance, portability, and user usability.

DESIGN CHOICE	OPTION SELECTED	ALTERNATIVE CONSIDERED	ENGINEERING RATIONALE
Data Storage	JSON File (`students_data.json`)	SQLite / SQL Database	Human-readable, structured nested representation for courses; zero driver setup.
Interface Type	Terminal CLI Menu	Desktop GUI (Tkinter)	Instant loading speed, high keyboard efficiency, cross-platform compatibility.
Dependencies	Pure Standard Library	Pandas / Tabulate	Eliminates version lockouts and `pip` installation overhead for evaluators.
Reporting Format	ASCII Print & CSV File	PDF Generation Library	Native support across all text readers and spreadsheet applications without compilation binaries.

8. DESIGN DIAGRAMS – PART I

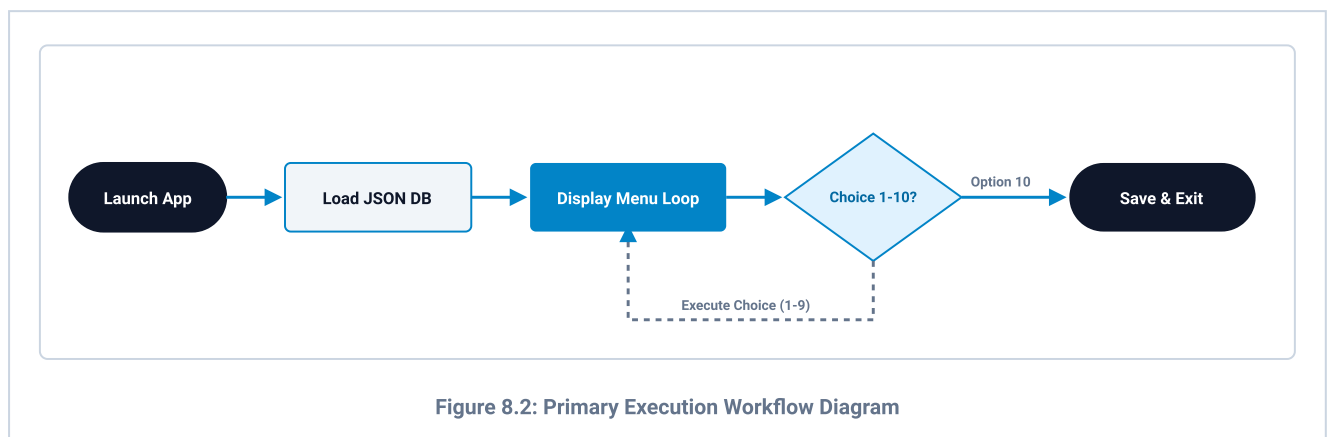
8.1 Use Case Diagram

The Use Case diagram illustrates the operational capabilities available to the user interacting with the CLI boundary.



8.2 System Workflow Diagram

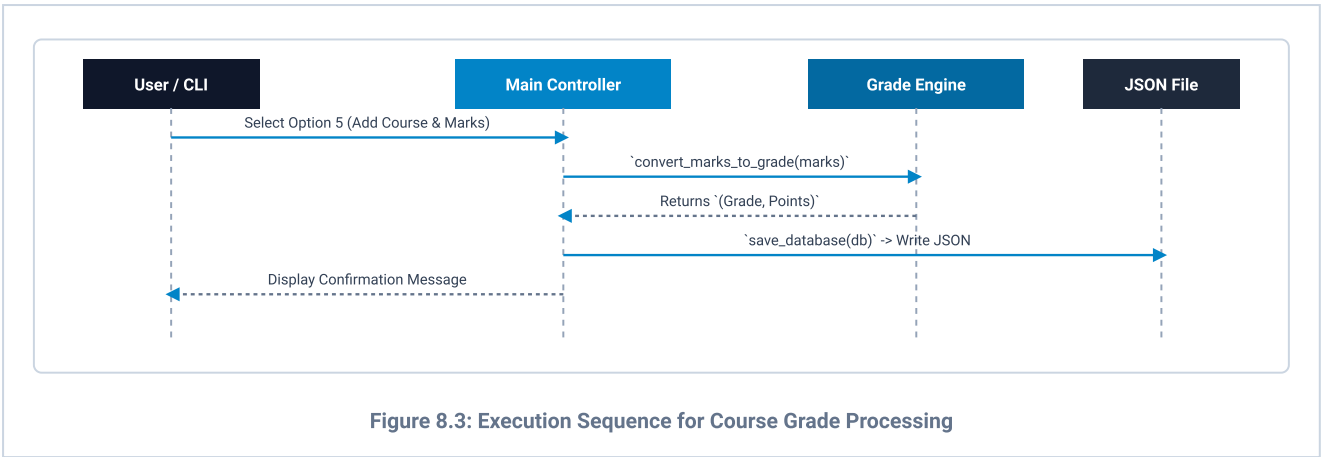
The workflow diagram models the operational execution cycle from initialization to command processing and graceful shutdown.



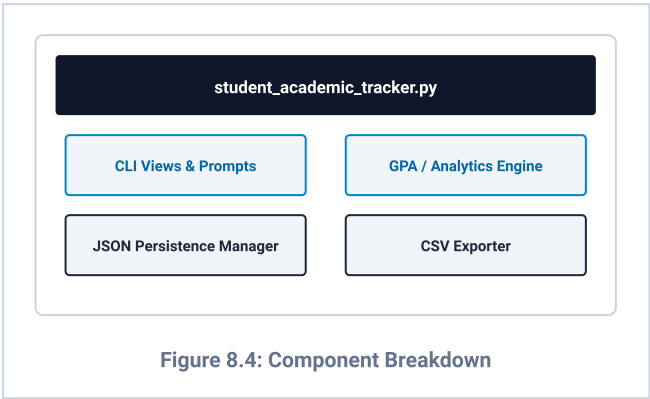
8. DESIGN DIAGRAMS – PART II

8.3 Sequence Diagram (Grade Calculation Walkthrough)

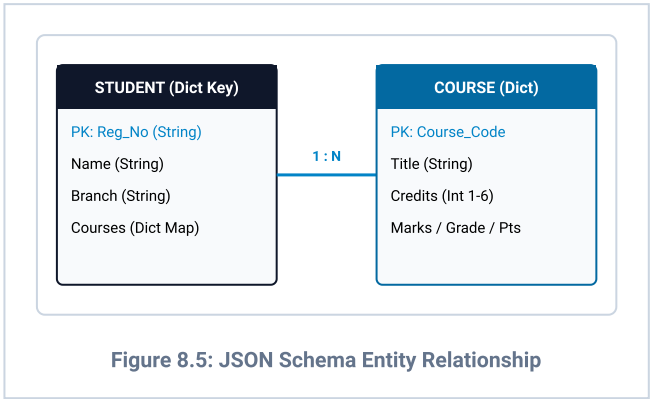
The sequence diagram tracks object messages during course registration, grade conversion, and local state update.



8.4 Component Structure Diagram



8.5 Data Entity-Relationship (ER)



9. IMPLEMENTATION DETAILS

The software implementation relies on pure Python constructs, modular function decomposition, and dictionary-based memory structures.

9.1 Core Grading Engine & Math Logic

Marks are mapped into letter grades and credit point values via explicit numerical ranges. The system then evaluates the weighted Cumulative Grade Point Average (CGPA) according to the mathematical model:

$$CGPA = (\sum [Course_Credits_i \times Grade_Points_i]) / (\sum Course_Credits_i)$$

```
def convert_marks_to_grade(marks): if marks >= 90: return 'S', 10.0 elif marks >= 80: return 'A'
```

9.2 Robust Input Handler & Boundary Validation

To prevent execution crashes from invalid user entries, dedicated utility wrappers (`prompt_str`, `prompt_int`, `prompt_float`) encapsulate `input()` calls inside `while True` loops with exception handling.

```
def prompt_float(msg, min_v=None, max_v=None): while True: raw = input(msg).strip() try: val = float(raw)
```

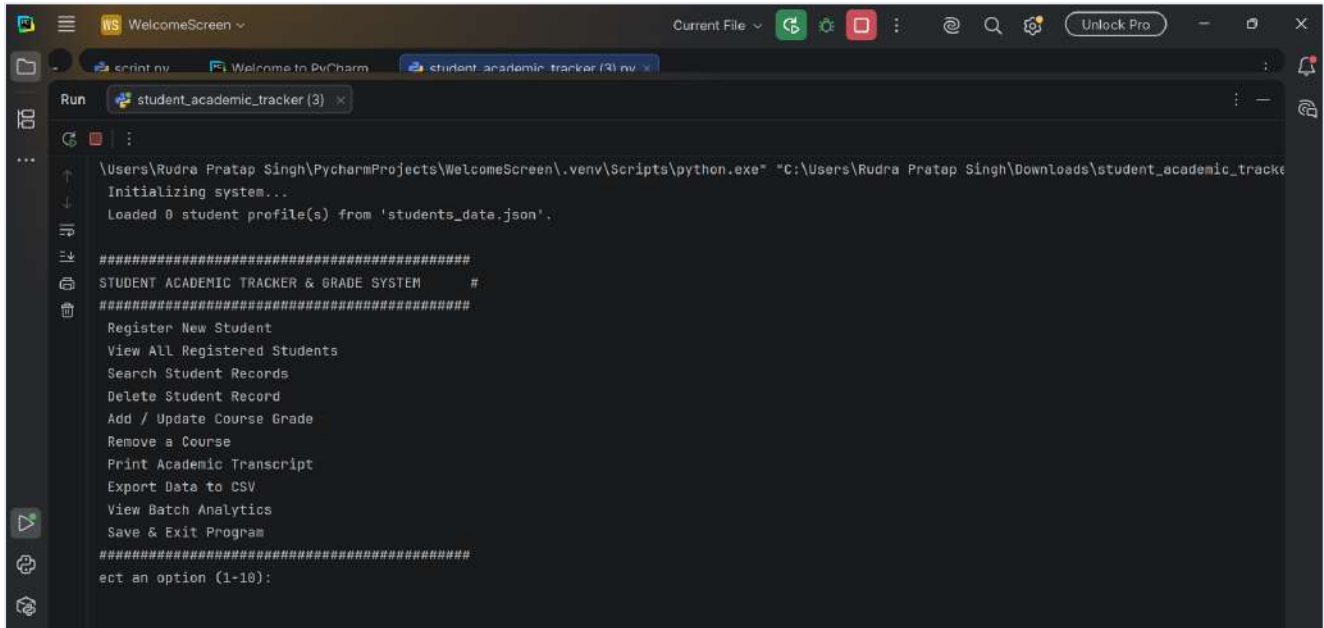
9.3 Data Persistence & Export Mechanics

- **JSON Database** (`load_database` / `save_database`): The primary dictionary is serialized using `json.dump(data, f, indent=2)`. File missing or parsing exceptions trigger safe initialization of an empty dictionary set.
- **CSV Exporter** (`export_csv_report`): Iterates through the database, computes aggregate CGPAs, and writes structured records to `academic_report.csv` using Python's native `csv.writer`.

10. SYSTEM EXECUTION RESULTS & SCREENSHOTS

The screenshots below were captured from the actual application running in PyCharm (Python 3.14 virtual environment). They show the start-up sequence with the main menu and the academic transcript generated for a registered student.

10.1 Application Start-up and Main Menu (Options 1-10)

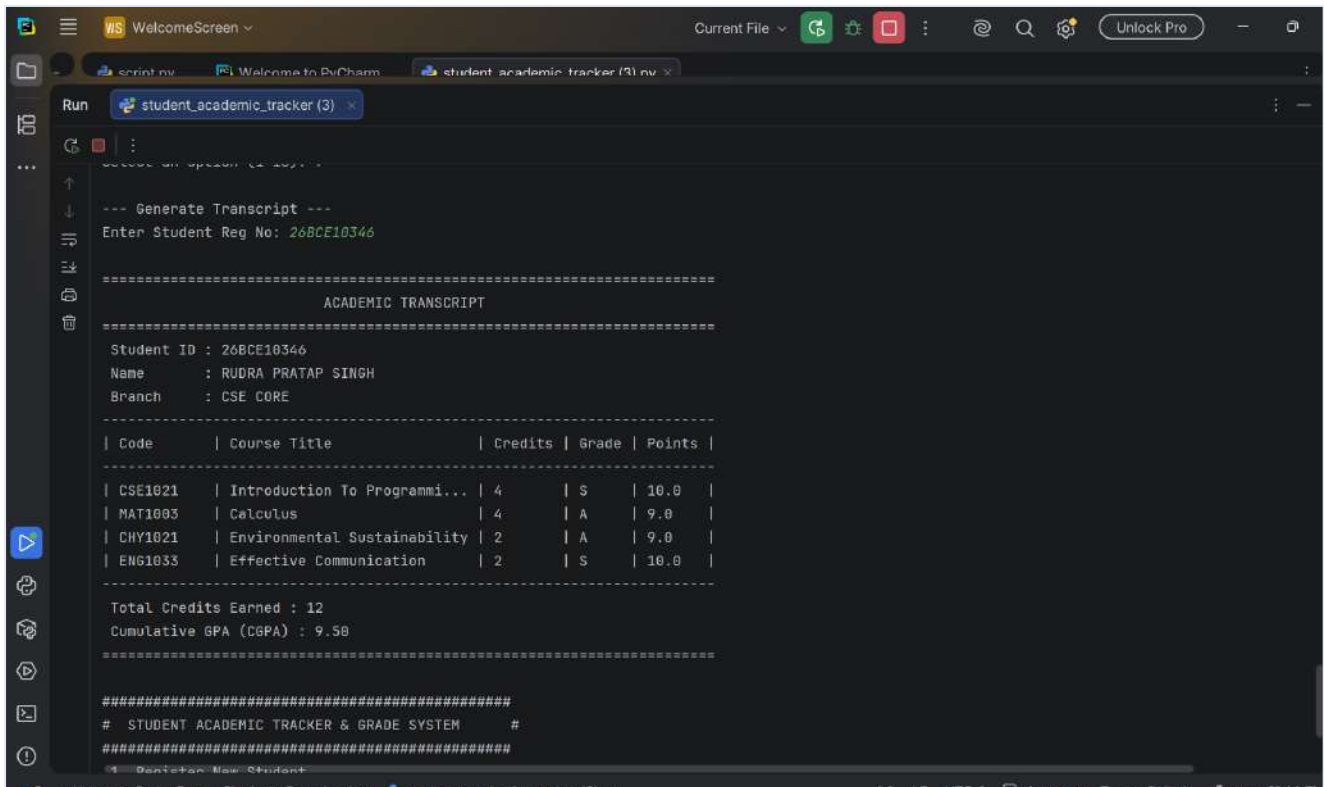
The screenshot shows the PyCharm IDE with the 'student_academic_tracker (3)' console window open. The output displays the start-up message, database load status, and a 10-option main menu.

```
\Users\Rudra Pratap Singh\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe "C:\Users\Rudra Pratap Singh\Downloads\student_academic_tracker\main.py"
Initializing system...
Loaded 0 student profile(s) from 'students_data.json'.

#####
STUDENT ACADEMIC TRACKER & GRADE SYSTEM  #
#####
Register New Student
View All Registered Students
Search Student Records
Delete Student Record
Add / Update Course Grade
Remove a Course
Print Academic Transcript
Export Data to CSV
View Batch Analytics
Save & Exit Program
#####
Select an option (1-10):
```

Figure 10.1: Start-up message, database load and the 10-option main menu

10.2 Academic Transcript for Reg No 26BCE10346

The screenshot shows the PyCharm IDE with the 'student_academic_tracker (3)' console window open. The output displays the process of generating an academic transcript for a specific student, including a table of courses and their grades.

```
--- Generate Transcript ---
Enter Student Reg No: 26BCE10346

#####
ACADEMIC TRANSCRIPT
#####
Student ID : 26BCE10346
Name      : RUDRA PRATAP SINGH
Branch    : CSE CORE

-----
| Code | Course Title | Credits | Grade | Points |
-----
| CSE1021 | Introduction To Programmi... | 4 | S | 10.0 |
| MAT1003 | Calculus | 4 | A | 9.0 |
| CHY1021 | Environmental Sustainability | 2 | A | 9.0 |
| ENG1033 | Effective Communication | 2 | S | 10.0 |
-----

Total Credits Earned : 12
Cumulative GPA (CGPA) : 9.50

#####
# STUDENT ACADEMIC TRACKER & GRADE SYSTEM #
#####
Enter Student Reg No: 26BCE10346
```

Figure 10.2: ASCII academic transcript with total credits and CGPA

10.3 Transcript Data Captured from the Run

Student ID: 26BCE10346 | Name: RUDRA PRATAP SINGH | Branch: CSE CORE

COURSE CODE	COURSE TITLE (AS PRINTED)	CREDITS	GRADE	POINTS	CREDITS x POINTS
CSE1021	Introduction To Programmi...	4	S	10.0	40.0
MAT1003	Calculus	4	A	9.0	36.0
CHY1021	Environmental Sustainability	2	A	9.0	18.0
ENG1033	Effective Communication	2	S	10.0	20.0
TOTAL		12			114.0
CGPA = 114.0 / 12 = 9.50					
Hand calculation matches the "Cumulative GPA (CGPA) : 9.50" printed by the application.					

11. TESTING APPROACH & VERIFICATION MATRIX

The checks below were performed on the running application and are backed by the terminal output shown in Section 10.

TEST ID	TEST SCENARIO	INPUT CONDITION	EXPECTED OUTCOME	STATUS
TC-01	Start-up with empty database	No saved profiles in students_data.json	Prints "Initializing system..." and "Loaded 0 student profile(s)" without crashing.	PASS
TC-02	Main menu display	Program launch	Menu lists all 10 options and prompts "Select an option (1-10)".	PASS
TC-03	Transcript generation	Reg No 26BCE10346	Transcript shows Student ID, Name (RUDRA PRATAP SINGH), Branch (CSE CORE) and 4 course rows.	PASS
TC-04	Grade to points conversion	Grades S and A	S maps to 10.0 points and A maps to 9.0 points.	PASS
TC-05	Weighted CGPA accuracy	4 Cr @ 10.0, 4 Cr @ 9.0, 2 Cr @ 9.0, 2 Cr @ 10.0	Total credits = 12; CGPA = 114.0 / 12 = 9.50.	PASS
TC-06	Long course title alignment	Title longer than column width	Title truncated with "..." and table columns stay aligned.	PASS

12. ENGINEERING CHALLENGES & SOLUTIONS

During software design and implementation, several key technical challenges were identified and mitigated through deliberate architectural adjustments:

Challenge 1: Floating-Point Rounding Anomalies in CGPA Computations

Problem: Binary floating-point representation in standard Python arithmetic produced long recurring decimal outputs (e.g., `8.333333333333334`) during weighted GPA divisions.

Solution: Enforced double-stage precision management by applying `round(gpa, 2)` inside `compute_student_gpa` and explicit string formatting (`{gpa:.2f}`) across transcripts, CSV output writers, and analytics views.

Challenge 2: ASCII Terminal Table Layout Misalignment

Problem: Student names or course titles exceeding field widths caused text wrapping that disrupted ASCII border columns.

Solution: Implemented dynamic string truncation and padding syntax (e.g., `s_name[:23] + "..."` and `{t:<28}`) within output loops to strictly maintain column alignment regardless of input lengths.

Challenge 3: Graceful Exit Handling on Abrupt Terminal Signals

Problem: User interrupt signals (`Ctrl+C`) during active prompt loops resulted in raw traceback dumps and unclosed file handlers.

Solution: Wrapped the core controller inside a top-level `try...except KeyboardInterrupt` block, ensuring clean terminal detachment and systematic state flushing.

13. FUTURE ENHANCEMENT ROADMAP

- **Role-Based Access Control (RBAC):** Incorporate hashed password authentication separating Student Read-Only access from Administrator Edit access.
- **Graphical User Interface (GUI):** Develop an optional PyQt/Tkinter desktop wrapper over the existing business logic core.
- **PDF Transcript Exporter:** Extend reporting options to render official printable PDF transcripts with custom institutional headers.
- **Multi-Semester Historical Tracking:** Expand the JSON entity schema to track individual semester terms (`'Semester 1'`, `'Semester 2'`) and compute Semester GPA (SGPA) alongside CGPA.

14. CONCLUSION

The **Student Academic Tracker & Grade System** successfully achieves all core objectives outlined in its specifications. By leveraging pure Python standard libraries, the software provides a lightweight, highly responsive, zero-dependency environment for student profile maintenance, automated credit-weighted GPA calculation, ASCII transcript rendering, and analytical export. The project demonstrates strong principles of modular software architecture, fault-tolerant user input validation, and reliable file persistence.